

Melqui Alexander Romero Veru

Harold Steven Sabogal Perez

Fundación Universitaria del Área Andina

ALEX YAHILTON TOVAR GONZALEZ

GESTIÓN DE BASES DE DATOS

04 de mayo del 2025

Contenido

Construcción del modelo relacional	6
Como respuesta de la IA obtuvimos lo siguiente:	6
👤 Contexto (Cliente)	6
📄 Requerimientos del sistema	7
Solución del problema	8
Herramientas:	8
Análisis y estructuración relacional de las entidades y atributos.	9
Estructuración de tipos de datos para cada atributo de las entidades	10
Desarrollo de la base de datos relacional modelo físico	11
Implementaciones	11
TRIGGER	11
INSERT:	18
VIEW - Para las Consultas:	19
Procedimiento	23
Según la definición de integridad, en los espacios en blanco completar el texto en inglés.....	25
Desarrollo del punto	25
Conclusiones	26
Evidencia Trabajo en Equipo	28

Introducción

El presente documento detalla el proceso de desarrollo llevado a cabo para la actividad evaluativa del Eje 2 del curso "Gestión de Bases de Datos". El objetivo principal fue diseñar, construir e implementar una base de datos relacional funcional, enfocada en un Sistema de Gestión de Seguridad Logística (SGSL) para una empresa ficticia, LogiSecure S.A.S., cuyos requerimientos fueron definidos mediante una interacción simulada con una inteligencia artificial actuando como cliente.

A lo largo de este informe, se expondrá el recorrido metodológico seguido, iniciando con la conceptualización del problema y la definición de requerimientos. Se detallará el diseño del modelo de datos, abarcando las fases conceptual, lógica y física, utilizando herramientas específicas como StarUML y DrawSQL. Posteriormente, se presentará la implementación práctica de la base de datos en SQL Server, incluyendo la creación de tablas, la definición de relaciones, y la implementación de elementos avanzados como Triggers para la automatización de alertas, Vistas para consultas seguras y Procedimientos Almacenados para encapsular lógica de negocio.

Además, se abordará un componente clave de la seguridad de bases de datos: la integridad de los datos, complementando un ejercicio práctico propuesto en la actividad. Este trabajo busca no solo cumplir con los requisitos académicos, sino también demostrar la aplicación práctica de los conceptos de gestión y seguridad de bases de datos en un escenario realista.

Objetivos

Objetivo General

Diseñar, construir e implementar una base de datos relacional segura y funcional para el Sistema de Gestión de Seguridad Logística (SGSL) de la empresa LogiSecure S.A.S., aplicando los conceptos y técnicas de modelado, gestión y seguridad de bases de datos aprendidos, y cumpliendo con los requerimientos especificados en la actividad evaluativa del Eje 2.

Objetivos Específicos

Elaborar el modelo de datos relacional completo (conceptual, lógico y físico) para el SGSL, identificando y definiendo claramente las entidades, atributos, relaciones, llaves primarias, llaves foráneas y tipos de datos pertinentes, utilizando herramientas de modelado como StarUML y DrawSQL.

Implementar la estructura física de la base de datos en el gestor SQL Server, traduciendo el modelo lógico a sentencias SQL para la creación de tablas, índices y restricciones de integridad referencial, asegurando la correcta representación del modelo diseñado.

Desarrollar e implementar componentes avanzados de la base de datos, tales como Triggers para la automatización de alertas de seguridad, Vistas para controlar el acceso a la información sensible y Procedimientos Almacenados para encapsular operaciones comunes, demostrando el uso adecuado de sentencias SQL y la aplicación de mecanismos de seguridad

Construcción del modelo relacional

Se debe construir un modelo relacional, la BD y crear copias de seguridad usando las sentencias SQL.

Para iniciar con la estructuración de la base de datos, comenzamos con la temática en la que se haría por lo que le solicitamos a la IA (ChatGPT) que actuará como un cliente que tiene una necesidad en particular, la cual pueda ser satisfecha con la base de datos relacional:

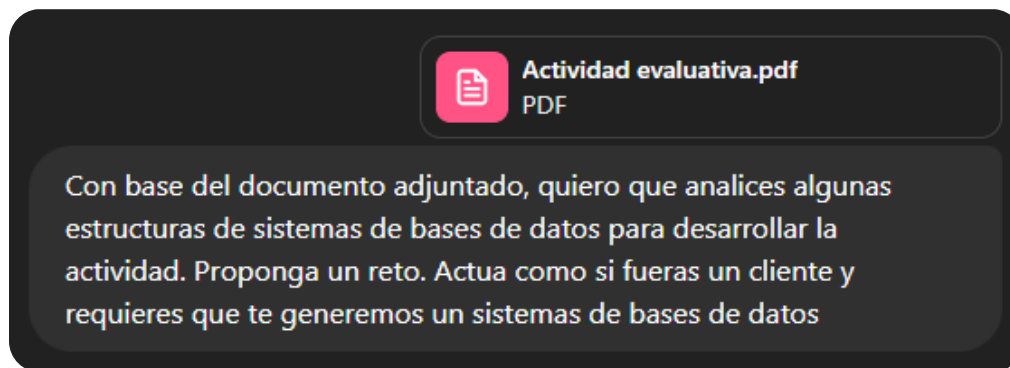


Figura 1, prompt solicitud de temática.

Como respuesta de la IA obtuvimos lo siguiente:

Contexto (Cliente)

Hola equipo

desarrollador,

Soy

la jefa de TI de LogiSecure

S.A.S., una empresa de logística que transporta

mercancía a nivel nacional. Necesitamos implementar un Sistema de Gestión de Seguridad

Logística

(SGSL) que permita:

- Controlar accesos de empleados,
- Rastrear eventos de seguridad,
- Registrar incidentes,
- Generar alertas automáticas,
- Y proteger los datos sensibles en nuestra base de datos.

Requerimientos del sistema

(base de datos)

1. Modelo relacional:

Debe contener como

mínimo las siguientes entidades:

- Empleados (con roles como conductor, supervisor, seguridad, etc.)
- Rutas (origen, destino, nivel de riesgo)
- Eventos (tipo: inspección, incidente, robo, parada no autorizada)
- Vehículos (tipo, placa, estado)
- Usuarios del sistema (con autenticación y roles de acceso)
- Historial de Accesos (con fecha, hora, IP, tipo de acceso)
- Alertas (automáticas por ciertos eventos)

2. Requisitos técnicos avanzados:

- ☒ Integridad referencial con llaves foráneas y validaciones.
- ☒ Vistas seguras que oculten columnas sensibles (como salario o ubicación en tiempo real).
- ☒ Procedimientos almacenados para registrar eventos y alertas.
- ☒ Triggers (disparadores) que lancen una alerta automáticamente cuando:

- Se registre un evento de tipo "robo"
- Un vehículo salga de ruta no autorizada
- ☒ Encriptación de datos sensibles (como contraseñas, tokens de acceso, ubicación).
- ☒ Backups automáticos mediante scripts o eventos programados.
- ☒ Roles y privilegios SQL según perfiles:
 - Administrador: Acceso total
 - Supervisor: Solo lectura de eventos y alertas
 - Seguridad: Registro de incidentes y accesos

Solución del problema

Una vez dado el problema iniciamos con la selección de las herramientas para la construcción del modelo relacional hemos utilizado las siguientes herramientas, además de agregar en qué modelo se iban a utilizar.

Herramientas:

- Star UML (Conceptual)
- Drawl SQL (Lógico)
- DBeaver, Beekeeper y Visual Studio Code (Físico)
- Gestor bases de datos: SQL Server

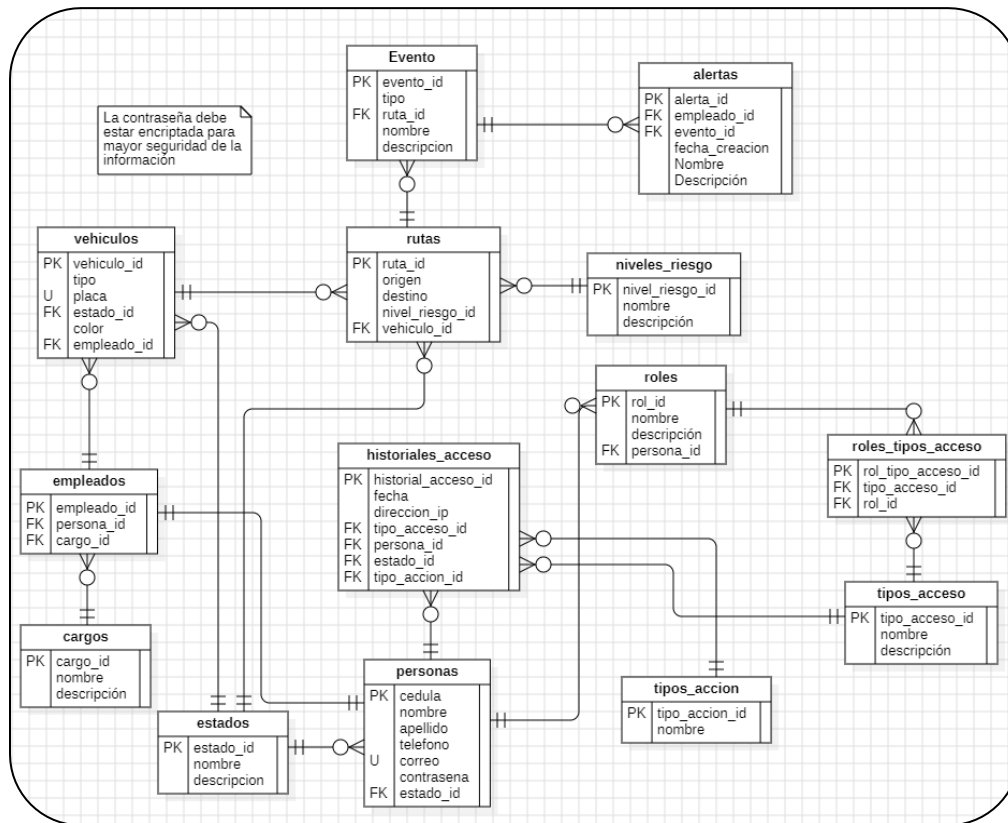


Figura 2, Modelo Conceptual utilizando StarUML.

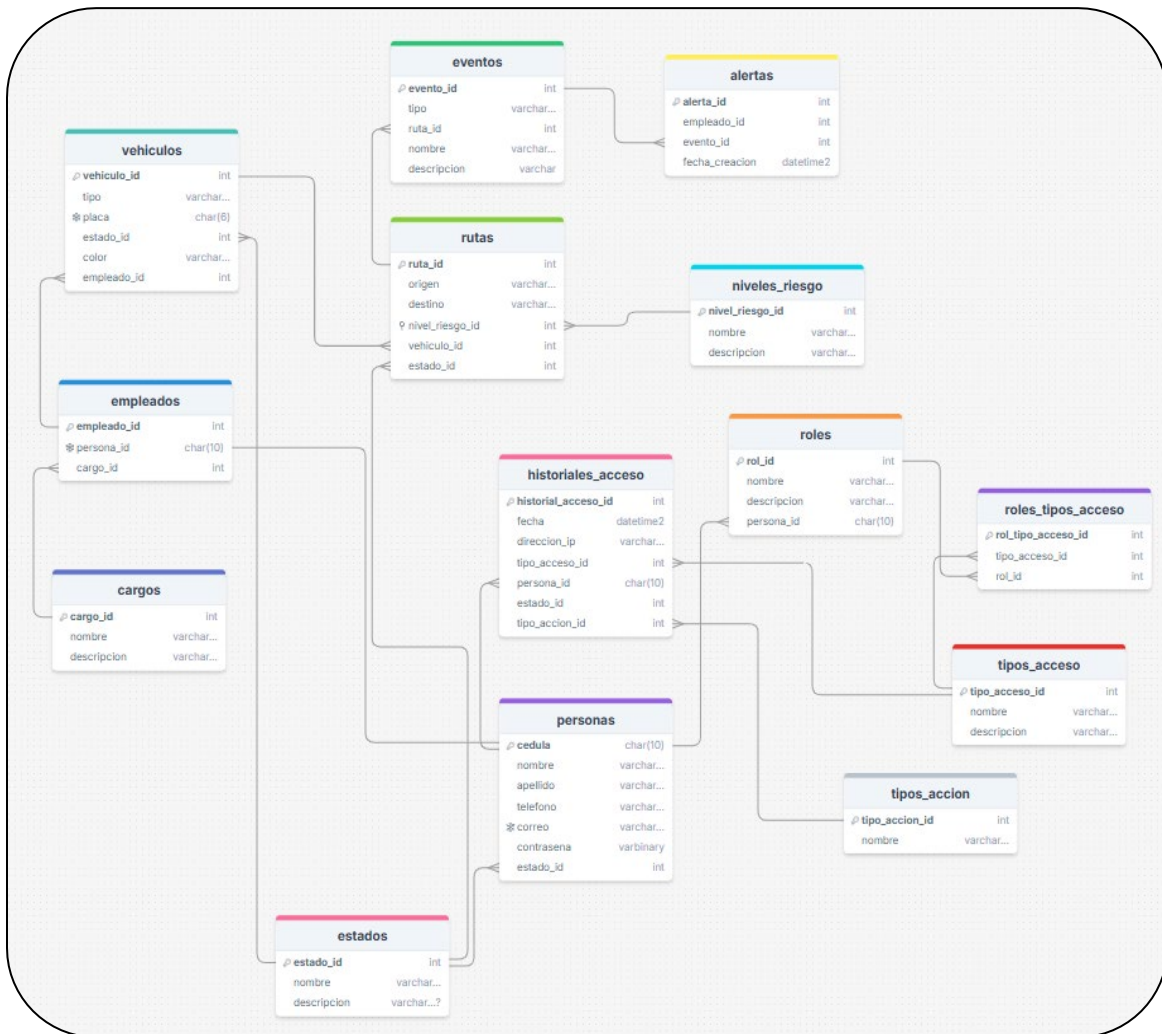


Figura 3, Modelo lógico utilizando DrawSQL

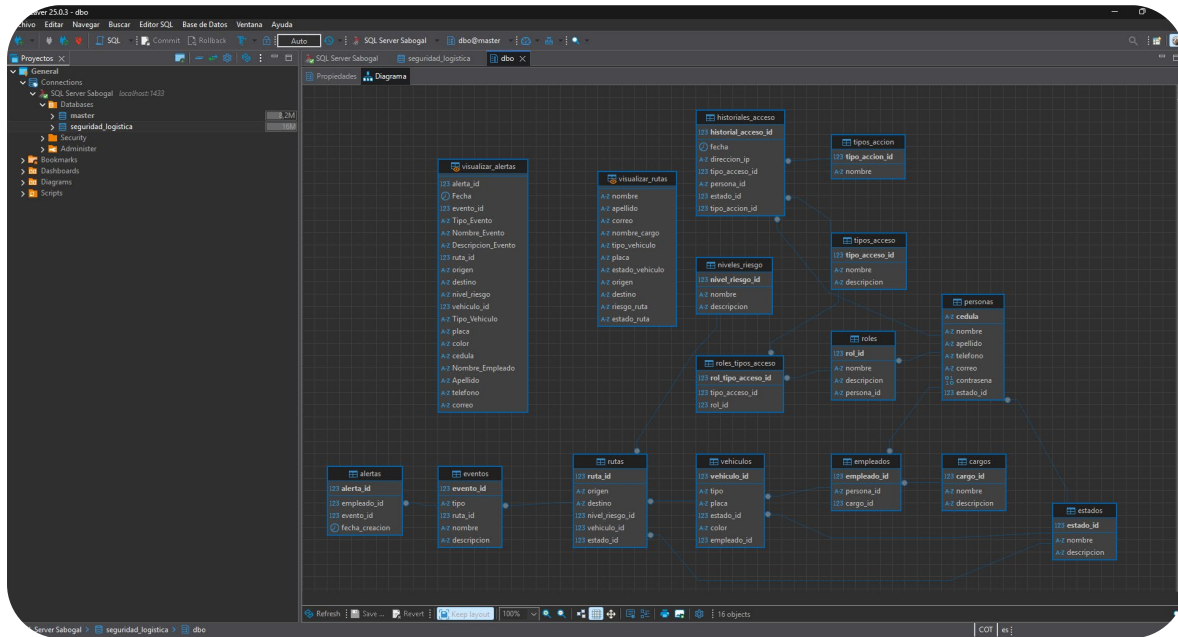


Figura 4, Diagrama del modelo lógico sacado con DBeaver

Implementaciones

TRIGGER

- “trg_insert_alerta_on_evento”

Funcionalidad:

Empleado → agrega Evento → rellena tabla:

Evento	
id_evento	int
Tipo	varchar(55)
id_ruta	int
Nombre	varchar(55)
Descripcion	text

Figura 5, Estructura tabla Evento.

El disparador → detecta el nuevo evento y se llena la tabla Alerta:

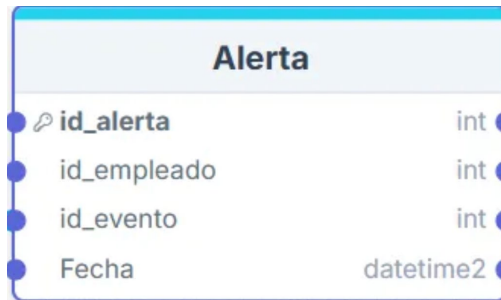


Figura 6, Estructura tabla Alerta.

El campo id_alerta se llena con → auto increment

El campo id_empleado → con ayuda de la tabla Ruta se agrega el campo id_empleado

El campo id_evento → el campo id_evento de la tabla Evento

El campo Fecha → Datetime

Trigger generado

Trigger

```

CREATE TRIGGER trg_insert_alerta_on_evento
ON "eventos"
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO "alertas" ("empleado_id", "evento_id", "fecha_creacion")
    SELECT
        V."empleado_id",          -- Obtenemos el empleado_id desde el vehiculo asociado a la ruta
        I."evento_id",           -- id_evento recién insertado
        GETDATE()                -- fecha actual del servidor
    FROM
        INSERTED I
    INNER JOIN "rutas" R ON R."ruta_id" = I."ruta_id"
    INNER JOIN "vehiculos" V ON V."vehiculo_id" = R."vehiculo_id";
END;
    
```

Figura 7, Trigger “trg_insert_alerta_on_evento”

- “trg_cambio_estado_ruta”

La lógica consiste en definir un desencadenante (*trigger*) que se ejecuta cuando se actualiza un registro en la tabla rutas. Al producirse esta actualización, se obtiene el nuevo valor de la columna estado_id. A partir de este nuevo estado, se realizan las validaciones correspondientes según los distintos identificadores de estado que puede tener una ruta. En función del estado asignado, se actualizan también los estados relacionados en las tablas persona y vehiculo.

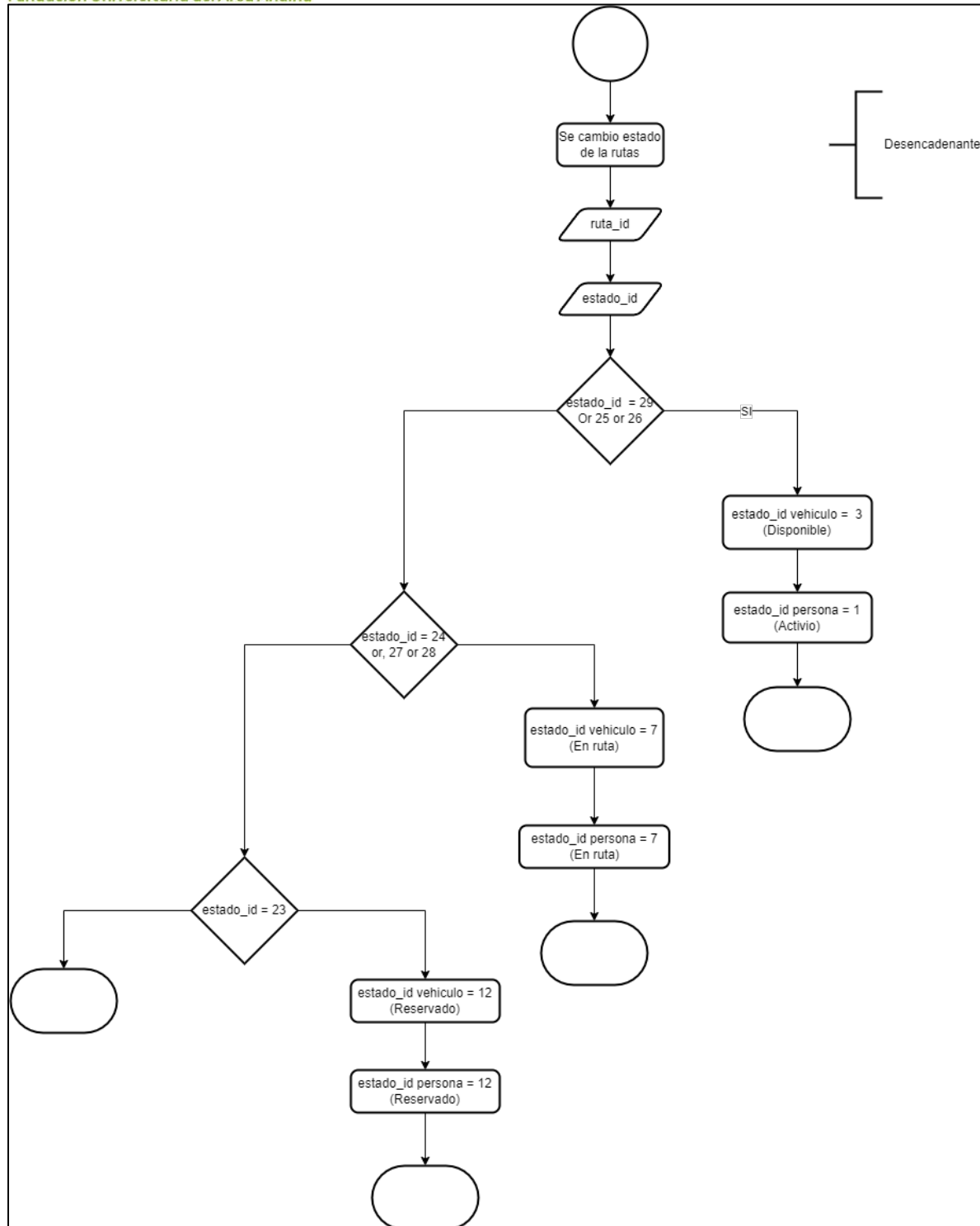
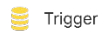


Figura 8, lógica estructurada para el trigger “trg_cambio_estado_ruta”



Trigger

```
CREATE TRIGGER trg_cambio_estado_ruta
ON "rutas"
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON; -- NO MUESTRE LAS TABLAS ALTERADAS

    DECLARE @estado_id INT = (SELECT estado_id FROM inserted);
    DECLARE @ruta_id INT = (SELECT ruta_id FROM inserted);
    DECLARE @vehiculo_id INT = (SELECT vehiculo_id FROM rutas WHERE ruta_id = @ruta_id);
    DECLARE @empleado_id INT = (SELECT empleado_id FROM vehiculos WHERE vehiculo_id = @vehiculo_id);
    DECLARE @persona_id CHAR(10) = (SELECT persona_id FROM empleados WHERE empleado_id = @empleado_id)

    IF @estado_id = 27 OR @estado_id = 28 OR @estado_id = 31
    BEGIN
        UPDATE vehiculos SET estado_id = 3
        WHERE vehiculos.vehiculo_id = @vehiculo_id;

        UPDATE personas SET estado_id = 1
        WHERE personas.cedula = @persona_id;
    END
    ELSE IF @estado_id = 26 OR @estado_id = 29 OR @estado_id = 30
    BEGIN
        UPDATE vehiculos SET estado_id = 7
        WHERE vehiculos.vehiculo_id = @vehiculo_id;

        UPDATE personas SET estado_id = 7
        WHERE personas.cedula = @persona_id;
    END
    ELSE IF @estado_id = 25
    BEGIN
        UPDATE vehiculos SET estado_id = 12
        WHERE vehiculos.vehiculo_id = @vehiculo_id;

        UPDATE personas SET estado_id = 12
        WHERE personas.cedula = @persona_id;
    END
END;
```

Figura 9, Trigger “trg_cambio_estado_ruta”

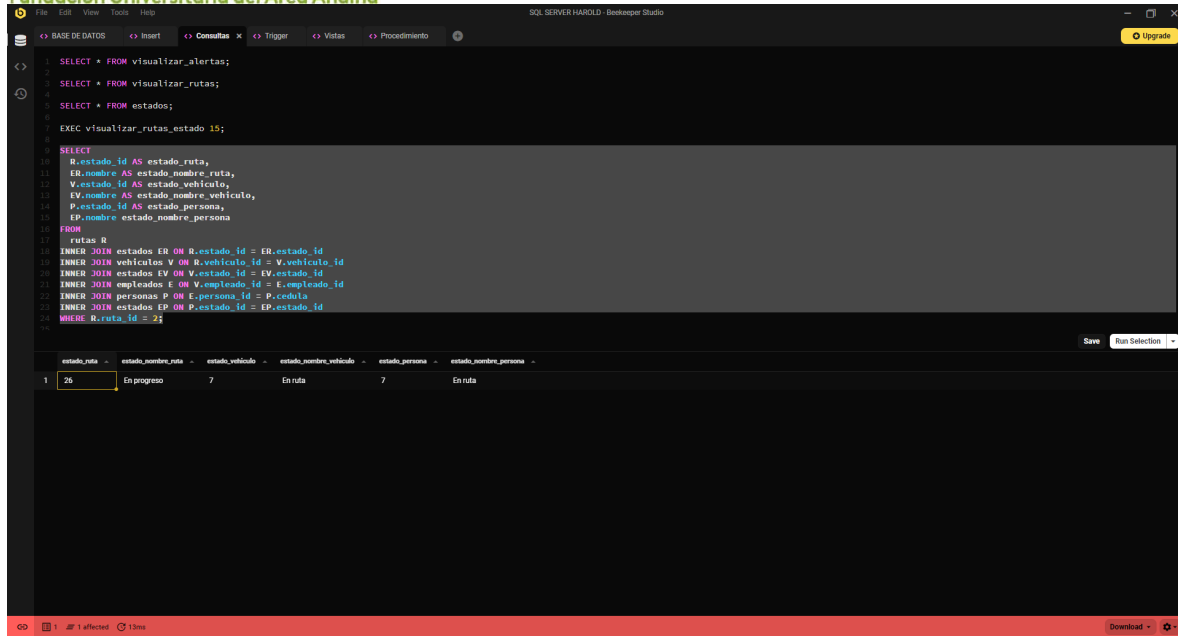


Figura 10, Visualización de estados antes de hacer uso del detonante del trigger

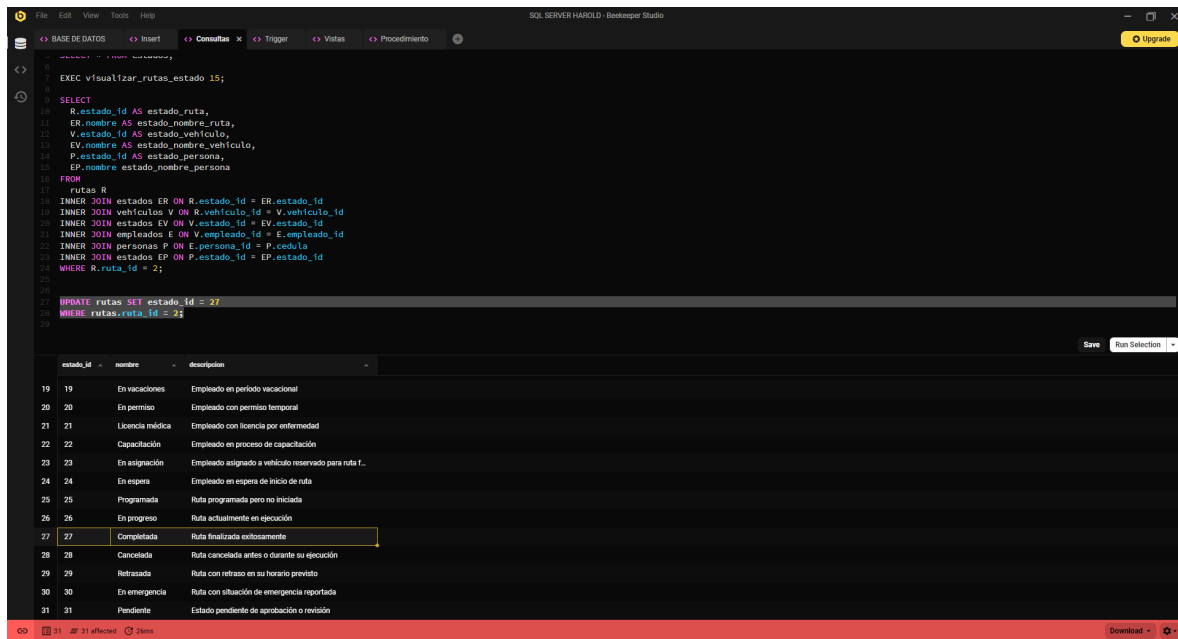


Figura 11, Visualización de estados, mostrando por el cual se va a cambiar

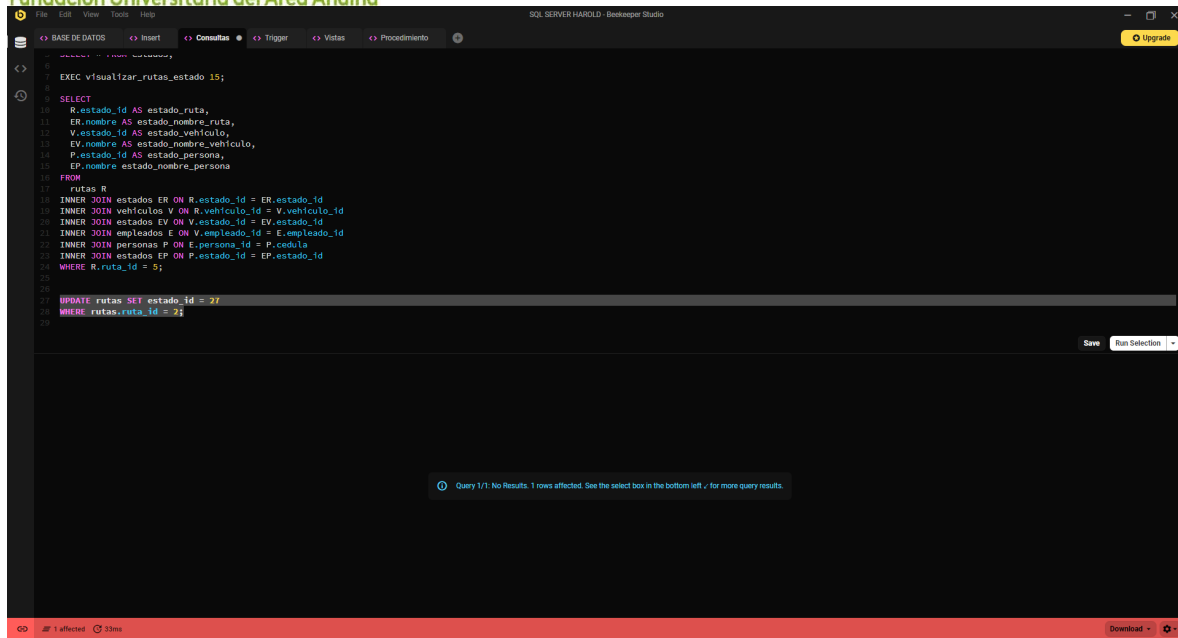


Figura 12, Visualización de la actualización exitosa

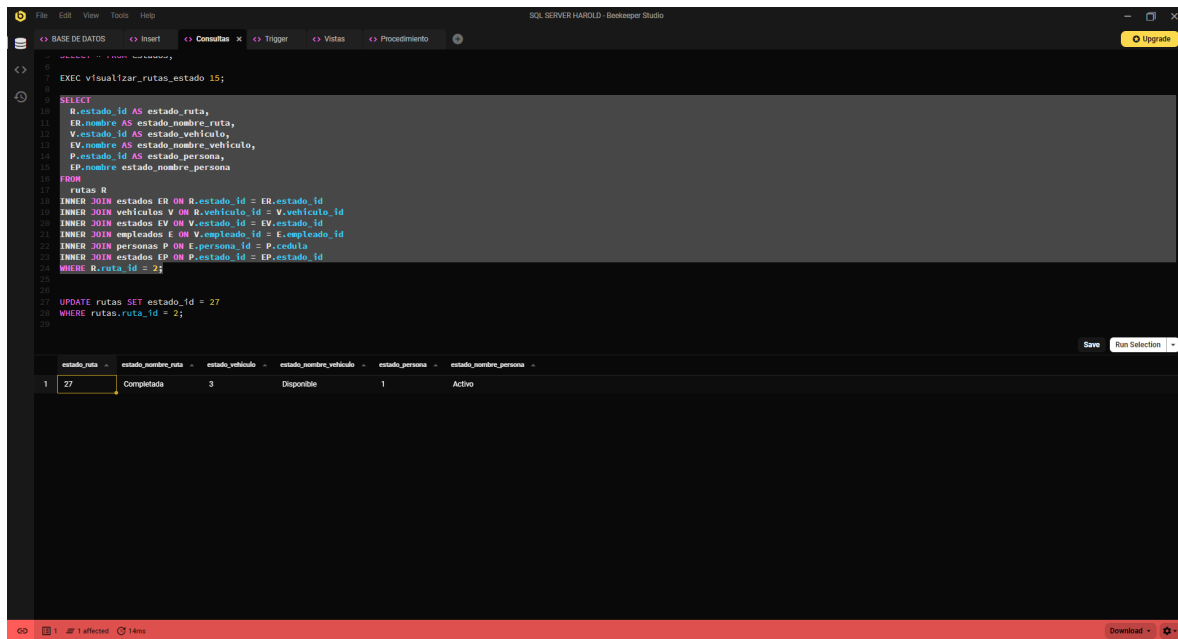


Figura 13, Visualización de estados despues de hacer uso del detonante del trigger

INSERT:

 INSERT

```
INSERT INTO personas (cedula, nombre, apellido, telefono, correo, contrasena, estado_id) VALUES
('1234567890', 'Juan', 'Pérez', '0991234567', 'juan.perez@empresa.com', CONVERT(VARBINARY(MAX),
'password123'), 1),
('2345678901', 'María', 'Gómez', '0987654321', 'maria.gomez@empresa.com', CONVERT(VARBINARY(MAX),
'securepass'), 1),
('3456789012', 'Carlos', 'Rodríguez', '0976543210', 'carlos.rod@empresa.com', CONVERT(VARBINARY(MAX),
'mypass123'), 1),
('4567890123', 'Ana', 'Martínez', '0965432109', 'ana.martinez@empresa.com', CONVERT(VARBINARY(MAX),
'anapass456'), 1),
('5678901234', 'Luis', 'García', '0954321098', 'luis.garcia@empresa.com', CONVERT(VARBINARY(MAX),
'luispass789'), 1);
```

Figura 14, Insert para tabla personas

- “visualizar_alertas”

 Vistas

```
CREATE VIEW visualizar_alertas
AS
SELECT
    A."alerta_id",
    A."fecha_creacion" AS "Fecha",
    E."evento_id",
    E."tipo" AS "Tipo_Evento",
    E."nombre" AS "Nombre_Evento",
    E."descripcion" AS "Descripcion_Evento",
    R."ruta_id",
    R."origen",
    R."destino",
    NR."nombre" AS "nivel_riesgo",
    V."vehiculo_id",
    V."tipo" AS "Tipo_Vehiculo",
    V."placa",
    V."color",
    P."cedula",
    P."nombre" AS "Nombre_Empleado",
    P."apellido" AS "Apellido",
    P."telefono",
    P."correo"
FROM
    "alertas" A
INNER JOIN "eventos" E ON A."evento_id" = E."evento_id"
INNER JOIN "rutas" R ON E."ruta_id" = R."ruta_id"
INNER JOIN "niveles_riesgo" NR ON R."nivel_riesgo_id" = NR."nivel_riesgo_id"
INNER JOIN "vehiculos" V ON R."vehiculo_id" = V."vehiculo_id"
INNER JOIN "empleados" EM ON A."empleado_id" = EM."empleado_id"
INNER JOIN "personas" P ON EM."persona_id" = P."cedula";
```

Figura 15, código de la vista “visualizar_alertas”

```
[
  {
    "id": 1,
    "fecha": "2025-05-01T20:29:32.543Z",
    "estado_id": 5,
    "tipo_evento": "Ruta alternativa",
    "causa": "Cierre de vía",
    "descripcion": "Derrumbe obligó a tomar ruta alternativa",
    "nivel_riesgo_id": 5,
    "origen": "Esmeraldas",
    "destino": "Quito",
    "riesgo": "Moderado",
    "vehiculo_id": 5,
    "tipo_vehiculo": "Trailer",
    "placa": "MN0345",
    "color": "Verde",
    "persona_id": "5678901234",
    "nombre": "Luis",
    "apellido": "García",
    "telefono": "0954321098",
    "correo": "luis.garcia@empresa.com"
  },
  {
    "id": 2,
    "fecha": "2025-05-01T20:29:32.543Z",
    "estado_id": 4,
    "tipo_evento": "Robo",
    "causa": "Intento de asalto",
    "descripcion": "Intento de robo frustrado por seguridad",
    "nivel_riesgo_id": 4,
    "origen": "Manta",
    "destino": "Esmeraldas",
    "riesgo": "Muy Alto",
    "vehiculo_id": 4,
    "tipo_vehiculo": "Camioneta",
    "placa": "JKL012",
    "color": "Negro",
    "persona_id": "4567890123",
    "nombre": "Ana",
    "apellido": "Martínez",
    "telefono": "0965432109",
    "correo": "ana.martinez@empresa.com"
  }
]
```

Figura 16, respuesta al ejecutar la vista “visualizar_alertas”

- visualizar_rutas



```
CREATE VIEW visualizar_rutas
AS
SELECT
    P."nombre",
    P."apellido",
    P."correo",
    C."nombre" AS "nombre_cargo",
    V."tipo" AS "tipo_vehículo",
    V."placa",
    EV."nombre" AS "estado_vehículo",
    R."origen",
    R."destino",
    NR."nombre" AS "riesgo_ruta",
    ER."nombre" AS "estado_ruta"
FROM
    "rutas" R
INNER JOIN "vehiculos" V ON R."vehiculo_id" = V."vehiculo_id"
INNER JOIN "empleados" E ON V."empleado_id" = E."empleado_id"
INNER JOIN "cargos" C ON E."cargo_id" = C."cargo_id"
INNER JOIN "personas" P ON E."persona_id" = P."cedula"
INNER JOIN "niveles_riesgo" NR ON R."nivel_riesgo_id" = NR."nivel_riesgo_id"
INNER JOIN "estados" EV ON V."estado_id" = EV."estado_id"
INNER JOIN "estados" ER ON R."estado_id" = ER."estado_id";
```

Figura 17, código de la vista “visualizar_rutas”

```
[
  {
    "nombre": "Juan",
    "apellido": "Pérez",
    "correo": "juan.perez@empresa.com",
    "rol": "Conductor",
    "tipo_vehiculo": "Camión",
    "placa": "ABC123",
    "estado_vehiculo": "Disponible",
    "origen": "Quito",
    "destino": "Guayaquil",
    "riesgo": "Moderado",
    "estado_evento": "Secuestrado"
  },
  {
    "nombre": "María",
    "apellido": "Gómez",
    "correo": "maria.gomez@empresa.com",
    "rol": "Supervisor",
    "tipo_vehiculo": "Furgoneta",
    "placa": "DEF456",
    "estado_vehiculo": "Disponible",
    "origen": "Guayaquil",
    "destino": "Cuenca",
    "riesgo": "Bajo",
    "estado_evento": "Secuestrado"
  },
  {
    "nombre": "Carlos",
    "apellido": "Rodríguez",
    "correo": "carlos.rod@empresa.com",
    "rol": "Coordinador",
    "tipo_vehiculo": "Pickup",
    "placa": "GHI789",
    "estado_vehiculo": "En reparación",
    "origen": "Cuenca",
    "destino": "Manta",
    "riesgo": "Alto",
    "estado_evento": "En robo"
  }
]
```

Figura 18, respuesta al ejecutar la vista “visualizar_rutas”

 Procedimiento

```
CREATE OR ALTER PROCEDURE visualizar_rutas_estado
    @estado INT = 1
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        DECLARE @variable_local INT = @estado;

        SELECT
            P."nombre",
            P."apellido",
            P."correo",
            C."nombre" AS "nombre_cargo",
            V."tipo" AS "tipo_vehiculo",
            V."placa",
            EV."nombre" AS "estado_vehiculo",
            R."origen",
            R."destino",
            NR."nombre" AS "riesgo_ruta",
            ER."nombre" AS "estado_ruta"
        FROM
            "rutas" R
        INNER JOIN "vehiculos" V ON R."vehiculo_id" = V."vehiculo_id"
        INNER JOIN "empleados" E ON V."empleado_id" = E."empleado_id"
        INNER JOIN "cargos" C ON E."cargo_id" = C."cargo_id"
        INNER JOIN "personas" P ON E."persona_id" = P."cedula"
        INNER JOIN "niveles_riesgo" NR ON R."nivel_riesgo_id" = NR."nivel_riesgo_id"
        INNER JOIN "estados" EV ON V."estado_id" = EV."estado_id"
        INNER JOIN "estados" ER ON R."estado_id" = ER."estado_id";
        WHERE ET."estado_id" = @estado;
    END TRY
    BEGIN CATCH
        DECLARE @ErrorMessage NVARCHAR(4000) = ERROR_MESSAGE();
        DECLARE @ErrorSeverity INT = ERROR_SEVERITY();
        DECLARE @ErrorState INT = ERROR_STATE();
        RAISERROR(@ErrorMessage, @ErrorSeverity, @ErrorState);
    END CATCH
END
```

Figura 19, código de procedimiento “visualizar_rutas_estado”



Respues Procedimiento Rutas

```
[
  {
    "nombre": "Juan",
    "apellido": "Pérez",
    "correo": "juan.perez@empresa.com",
    "rol": "Conductor",
    "tipo_vehiculo": "Camión",
    "placa": "ABC123",
    "origen": "Quito",
    "destino": "Guayaquil",
    "riesgo": "Moderado",
    "estado_evento": "Completada"
  },
  {
    "nombre": "María",
    "apellido": "Gómez",
    "correo": "maria.gomez@empresa.com",
    "rol": "Supervisor",
    "tipo_vehiculo": "Furgoneta",
    "placa": "DEF456",
    "origen": "Guayaquil",
    "destino": "Cuenca",
    "riesgo": "Bajo",
    "estado_evento": "Completada"
  }
]
```

Figura 20, respuesta al ejecutar procedimiento “visualizar_rutas_estado”

Según la definición de integridad, en los espacios en blanco completar el texto en inglés.

Integridad: es la garantía de que la información a la que se accede no ha sido alterada y que lo que se lee allí es exactamente lo que se pretende. Lo que la integridad nos sugiere es que la información a la que accedemos es verdaderamente fiable, que podemos aceptarla como un hecho. Pero sabemos que la información puede perder su integridad con una intervención maliciosa. Por ejemplo, cuando un hacker entra en el sistema de una universidad y cambia las calificaciones a conveniencia.

Sin embargo, la integridad también puede perderse sin querer. Un ejemplo de esto es cuando alguien autorizado por error borra o cambia cualquier archivo de la información, o cuando debido a un fallo de alimentación, el ordenador corrompe un archivo, y así sucesivamente.

Integrity: It is the guarantee that the information accessed has not been _____ and that what is read there is _____ what is intended. What _____ [suggests to us is that the information we access is truly reliable, that we can accept it as a fact. But we know that information can lose its integrity with _____ intervention. For example, when a hacker enters a university's system and changes the _____ at convenience.

However, _____ can also be _____ lost. An example of this is when someone authorized by mistake _____ or changes any file in the _____, or when due to a power failure, the computer _____ a file, and so on.

Figura 21, Imagen actividad ingles

Desarrollo del punto

Integrity: It is the guarantee that the information accessed has not been **altered** and that what is read there is **exactly** what is intended. What **integrity** suggests to us is that the information we access is truly reliable, that we can accept it as a fact. But we know that information can lose its integrity with **malicious** intervention. For example, when a hacker enters a university's system and changes the **grades** at convenience.

However, **integrity** can also be **unintentionally** lost. An example of this is when someone authorized by mistake **deletes** or changes any file in the **information**, or when due to a power failure, the computer **corrupts** a file, and so on.

Conclusiones

La realización de esta actividad permitió consolidar y aplicar de manera práctica los conocimientos adquiridos en el curso de Gestión de Bases de Datos, específicamente en el diseño, implementación y aseguramiento de una base de datos relacional para un Sistema de Gestión de Seguridad Logística (SGSL).

Modelado Efectivo: Se logró traducir exitosamente los requerimientos funcionales del SGSL en un modelo de datos relacional coherente y bien estructurado (conceptual, lógico y físico). La utilización de herramientas como StarUML y DrawSQL facilitó la visualización y refinamiento del diseño, asegurando la correcta identificación de entidades, atributos y relaciones fundamentales para el sistema.

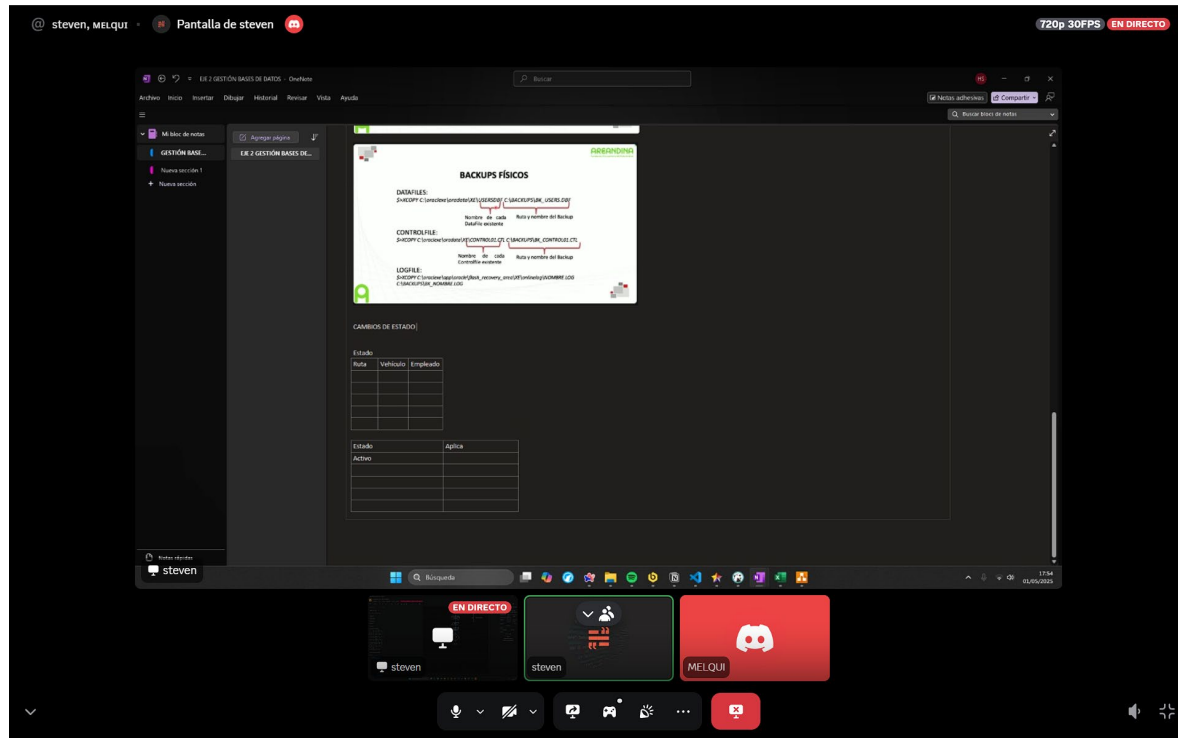
Implementación Exitosa en SQL Server: La fase de implementación física demostró la capacidad de traducir el modelo lógico a código SQL funcional. Se crearon las tablas, se establecieron las relaciones mediante llaves foráneas y se aplicaron restricciones de integridad, sentando una base sólida y confiable para el almacenamiento y gestión de los datos del SGSL.

Aplicación de Funcionalidades Avanzadas: La implementación de Triggers para la generación automática de alertas ante eventos específicos (como robos o desvíos de ruta), Vistas para proteger información sensible y Procedimientos Almacenados para operaciones recurrentes, evidenció la comprensión y el manejo de características avanzadas de SQL Server. Estos elementos no solo cumplen los requisitos técnicos, sino que añaden valor funcional y de seguridad significativo a la base de datos.

Comprensión de la Seguridad e Integridad: El desarrollo del proyecto y la resolución del ejercicio sobre integridad de datos reforzaron la importancia crítica de estos conceptos en la gestión de bases de datos. Se comprendió cómo la integridad referencial, la encriptación

(simulada en el requerimiento de contraseñas) y el control de acceso mediante roles y vistas son esenciales para garantizar la fiabilidad y confidencialidad de la información.

Evidencia Trabajo en Equipo

[illegible]

AREANDINA
Fundación Universitaria del Área Andina

Alerta

id_alerta	int
id_empleado	int
id_evento	int
Fecha	datetime2

Figura 5. Estructura tabla Alerta.

El campo id_alerta se llena con -> auto increment

El campo id_empleado -> con ayuda de la tabla Ruta se agrega el campo id_empleado

El campo id_evento -> el campo id_evento de la tabla Evento

El campo Fecha -> Datetime

```

1 CREATE TRIGGER trg_insert_alerta_on_evento
2 ON "eventos"
3 AFTER INSERT
4 AS
5 BEGIN
6     SET NOCOUNT ON;
7
8     INSERT INTO "alertas" ("empleado_id", "evento_id", "fecha_creacion")
9     SELECT
10         V."empleado_id",           -- Obtenemos el empleado_id desde el vertice asociado a la ruta
11         I."evento_id",            -- Obtenemos el evento_id desde el vertice asociado a la ruta
12         GETDATE()                -- Fecha actual del servidor
13     FROM
14         INSERTED I
15     INNER JOIN "rutas" R ON R."ruta_id" = I."ruta_id"
16     INNER JOIN "vehiculos" V ON V."vehiculo_id" = R."vehiculo_id";

```

Página 10 de 13 640 palabras Español (Colombia) Predicciones de texto: activado Accesibilidad: todo correcto

Notas durante el desarrollo de la actividad:

<https://www.notion.so/stevensabogal/EJE-2-1e57c01f2352804e8a03ce98f9a382c4?pvs=4>

Referencias

OpenAI (ChatGPT). (2025, Mayo). Comunicación personal (Generación de temática y requerimientos iniciales para el Sistema de Gestión de Seguridad Logística, basado en el análisis del documento "Actividad evaluativa.pdf").

Microsoft. (s.f.). Documentación de SQL Server. Recuperado el 4 de mayo de 2025, de

<https://learn.microsoft.com/es-es/sql/?view=sql-server-ver16>

StarUML. (s.f.). Herramienta utilizada para el Modelado Conceptual.

DrawSQL. (s.f.). Herramienta utilizada para el Modelado Lógico.

DBeaver. (s.f.). Herramienta utilizada para el Desarrollo Físico y Gestión de la Base de Datos.

Beekeeper Studio. (s.f.). Herramienta utilizada para el Desarrollo Físico y Gestión de la Base de Datos.

Visual Studio Code. (s.f.). Editor de código utilizado para el Desarrollo Físico y Gestión de scripts SQL.

Microsoft SQL Server. (s.f.). Sistema Gestor de Bases de Datos (SGBD) utilizado para la implementación.